الجمهورية الجزائرية الديمقراطية الشعبية
People's Democratic Republic of Algeria
وزارة التعليم العالي والبحث العلمي
Ministry of Higher Education and Scientific Research
المدرسة العليا للإعلام الآلي 8 ماي 5491 - سيدي بلعباس
Higher School of Computer Science
8 Mai 1945 – Sidi Bel Abbes



Master's Thesis

To obtain the diploma of **Master**
Field: **Computer Science**
Specialty: **Computer Systems Engineering (CIS)**

Theme

Towards Trustworthy SBOM Recovery from Compiled Android Applications

An Evidence-Based Component Attribution Framework

Presented by
Fendi Mohamed

Submission Date: **06/07/2026**

In front of the jury composed of

Dr. ANANI Djihed	Jury President
Dr. SERIAI Abdelhak Djamel	Supervisor
Dr. BABA AHMED Manel	Co-Supervisor
Dr. NAIB Rabiaa	Examiner

Academic Year: 2025–2026

Dedication

This work is dedicated to my parents,

to my dear sister and brother,

to my entire family,

and to the friends who accompanied and supported me throughout my journey, from elementary school to graduation.

Acknowledgements

First and foremost, I thank God for granting me the strength, patience, and perseverance necessary to complete this work and reach this important stage of my academic journey.

I express my deepest gratitude to my parents and my entire family. Their constant encouragement, sacrifices, trust, and unconditional support have shaped the person I am today. This achievement would not have been possible without the values they taught me and the confidence they placed in me.

I am sincerely grateful to Abdelhak Djamel Seriai and Manel Baba Ahmed for their guidance, availability, constructive feedback, and valuable advice throughout this project. Their support helped me address both the technical and research challenges that shaped this thesis.

I also extend my appreciation to the Higher School of Computer Science of Sidi Bel Abbès (ESI-SBA), its teachers, and its staff for the knowledge, academic environment, and opportunities provided throughout my studies. The education I received at the school has been essential to my growth as a computer science student and researcher.

Finally, I warmly thank all my friends who accompanied me from elementary school through graduation. Their friendship, encouragement, shared efforts, and presence made the difficult moments easier and the successful moments more meaningful. To everyone who contributed, directly or indirectly, to this journey, I offer my heartfelt thanks.

Abstract

Software Bills of Materials are increasingly used to support software supply-chain transparency and vulnerability management. However, a syntactically valid SBOM is not automatically trustworthy, particularly when it is reconstructed from a compiled Android application without source code or dependency manifests. Compilation, optimization, obfuscation, artifact-family aliases, reference-database gaps, and version similarity can weaken or contradict the evidence supporting a component claim.

This thesis studies trustworthy SBOM recovery as an evidence-based software component attribution problem. It organizes existing Android third-party library and binary component-recovery research into taxonomies of techniques, evidence sources, transformation threats, and evaluation practices. Building on the resulting research gaps, it proposes a conceptual framework in which each recovered claim records component identity, version or version set, evidence provenance, evidence quality, transformation risk, confidence, ambiguity, validation class, and permitted downstream security interpretation.

An implemented APK-to-SBOM system is used as a bounded case study. Its controlled, clean and partial source-validated, source-known upper-bound, experiment-only, report-only, and boundary evidence demonstrates why library identity, exact version, reference coverage, matcher recall, and vulnerability authority must remain separate. A case-study evaluation confirms that the framework preserves these evidence lanes and blocks authority increases when apparently beneficial changes fail ambiguity, control, or promotion gates. The proposed framework therefore favors auditable partial knowledge and explicit abstention over unsupported certainty, and defines a research agenda for calibrated evidence fusion, provenance-aware SBOMs, realistic benchmarks, and native or dynamically loaded component recovery.

Keywords — software bill of materials, software supply chain, Android, component attribution, provenance, evidence fusion, trustworthiness, uncertainty, vulnerability management.

Résumé

Les nomenclatures logicielles, ou SBOM, sont de plus en plus utilisées pour renforcer la transparence de la chaîne d'approvisionnement logicielle et la gestion des vulnérabilités. Toutefois, une SBOM syntaxiquement valide n'est pas automatiquement digne de confiance, en particulier lorsqu'elle est reconstruite à partir d'une application Android compilée, sans accès au code source ni aux manifestes de dépendances. La compilation, l'optimisation, l'obfuscation, les alias de familles d'artefacts, les lacunes des bases de références et la similarité entre versions peuvent affaiblir ou contredire les preuves qui soutiennent une déclaration de composant.

Ce mémoire étudie la reconstruction fiable de SBOM comme un problème d'attribution de composants logiciels fondé sur les preuves. Il organise les travaux existants sur la détection de bibliothèques tierces Android et la récupération de composants binaires selon des taxonomies de techniques, de sources de preuves, de menaces de transformation et de pratiques d'évaluation. À partir des lacunes ainsi mises en évidence, il propose un cadre conceptuel dans lequel chaque déclaration reconstruite conserve l'identité du composant, la version ou l'ensemble de versions, la provenance des preuves, leur qualité, le risque de transformation, le niveau de confiance, l'ambiguïté, la classe de validation et l'interprétation de sécurité autorisée en aval.

Un système d'analyse APK-vers-SBOM implémenté sert d'étude de cas délimitée. Ses preuves contrôlées, propres et partielles validées par les sources, ses bornes supérieures connues par les sources, ainsi que ses résultats expérimentaux, indicatifs et limites, montrent pourquoi l'identité de bibliothèque, la version exacte, la couverture des références, le rappel du moteur de correspondance et l'autorité des vulnérabilités doivent rester séparés. Le cadre est évalué par l'étude de cas : il préserve ces catégories de preuves et empêche toute augmentation d'autorité lorsqu'une modification apparemment bénéfique échoue aux critères d'ambiguïté, de contrôle ou de promotion. Le cadre proposé privilégie donc une connaissance partielle auditable et l'abstention explicite plutôt qu'une certitude non étayée, et définit un programme de recherche pour la fusion calibrée des preuves, les SBOM sensibles à la provenance, les bancs d'essai réalistes et la récupération de composants natifs ou chargés dynamiquement.

Mots-clés — nomenclature logicielle, chaîne d'approvisionnement logicielle, Android, attribution de composants, provenance, fusion de preuves, confiance, incertitude, gestion des vulnérabilités.

ملخص

تُستعمل قوائم مكونات البرمجيات، أو SBOM، بشكل متزايد لدعم شفافية سلسلة توريد البرمجيات وإدارة الثغرات. غير أن وجود SBOM صحيحة من الناحية التركيبية لا يعني بالضرورة أنها موثوقة، خاصة عندما يعاد بناؤها من تطبيق أندرويد مترجم دون الوصول إلى الشفرة المصدرية أو ملفات بيان الاعتماديات. فقد تؤدي الترجمة، والتحسين، والتعمية، وأسماء عائلات القطع البرمجية، ونقص قواعد المراجع، والتشابه بين الإصدارات إلى إضعاف الأدلة التي تدعم ادعاء وجود مكون معين أو إلى مناقضتها.

يدرس هذا العمل استرجاع SBOM الموثوقة بوصفه مسألة إسناد مكونات برمجية قائمة على الأدلة. وينظم الأبحاث السابقة حول كشف المكتبات الخارجية في أندرويد واسترجاع المكونات الثنائية ضمن تصنيفات للتقنيات، ومصادر الأدلة، ومخاطر التحويل، وممارسات التقييم. وبناء على الفجوات الناتجة، يقترح إطارا مفهوميا يسجل لكل ادعاء مسترجع هوية المكون، والإصدار أو مجموعة الإصدارات، ومصدر الأدلة، وجودتها، ومخاطر التحويل، ومستوى الثقة، والغموض، وفئة التحقق، والتفسير الأمني المسموح به لاحقا.

يُستخدم نظام مشروع نهاية الدراسة لتحليل APK وإنتاج SBOM كدراسة حالة محدودة. وتبين أدلته المضبوطة، والنتائج النظيفة والجزئية المحققة بالمصدر، والحدود العليا المعروفة بالمصدر، والنتائج التجريبية أو الإرشادية أو الحدية، ضرورة الفصل بين هوية المكتبة، والإصدار الدقيق، وتغطية المراجع، واسترجاع المطابق، وسلطة ادعاءات الثغرات. ويؤكد تقييم دراسة الحالة أن الإطار يحافظ على مسارات الأدلة هذه ويمنع رفع سلطة الادعاء عندما تفشل التغييرات التي تبدو مفيدة في اختبارات الغموض أو الضبط أو الترقية. لذلك يفضل الإطار المقترح معرفة جزئية قابلة للتدقيق وامتناعا صريحا عن الحكم بدلا من يقين غير مدعوم، كما يحدد برنامجا بحثيا لدمج الأدلة بصورة معايرة، ولبناء SBOM واعية بالمصدر، ولاختبارات واقعية، ولاسترجاع المكونات الأصلية أو المحملة ديناميكيا.

الكلمات المفتاحية — قائمة مكونات البرمجيات، سلسلة توريد البرمجيات، أندرويد، إسناد المكونات، مصدر الأدلة، دمج الأدلة، الثقة، عدم اليقين، إدارة الثغرات.

Contents

Dedication i

Acknowledgements ii

Abstract iii

Résumé iv

ملخص v

Contents vi

List of Figures ix

List of Tables x

List of Acronyms xi

I General Introduction 1

1 General Introduction 2

1.1 Context 2
1.2 Problem Statement 2
1.3 Motivation 3
1.4 Research Questions 3
1.5 Objectives 4
1.6 Scope 4
1.7 Thesis Organization 5

II Background 6

2 Background 7

2.1 Introduction 7

2.2 Software Supply Chains and SBOMs . . . 7
2.2.1 Authoritative and Reconstructed SBOMs . . . 7
2.3 Android Applications and Build Cycle . . . 8
2.3.1 Application Components . . . 8
2.3.2 Development, Build, Packaging, and Release . . . 8
2.3.3 APK Structure and DEX . . . 9
2.3.4 Libraries and Dependencies . . . 10
2.3.5 Static and Dynamic Analysis . . . 10
2.4 Trust, Evidence, Provenance, and Uncertainty . . . 11
2.4.1 Evidence and Claims . . . 11
2.4.2 Provenance and Independence . . . 11
2.4.3 Uncertainty and Abstention . . . 11
2.5 Android Compiled-Artifact Context . . . 11
2.6 Software Component Attribution . . . 12
2.6.1 Identity, Version, and Security Authority . . . 12
2.6.2 Reference Dependence . . . 13
2.7 Evidence Sources in APKs . . . 13
2.8 Conclusion . . . 14

III State of the Art 15

3 State of the Art and Taxonomy 16
3.1 Introduction . . . 16
3.2 Taxonomy of Component-Recovery Techniques . . . 17
3.3 Taxonomy of Evidence Sources . . . 18
3.4 Taxonomy of Transformation Threats . . . 20
3.5 Representative Android TPL Approaches . . . 21
3.5.1 From Profile Matching to Structural Boundaries . . . 21
3.5.2 Version-Aware and Vulnerability-Oriented Pipelines . . . 22
3.5.3 Learning-Based Presence Detection . . . 22
3.5.4 Empirical Assessments and Cross-Tool Studies . . . 22
3.6 Evaluation Practices and Ground Truth . . . 23
3.7 Comparative Analysis . . . 24
3.8 Limitations of Existing Research . . . 26
3.9 Research Gap . . . 27
3.10 Conclusion . . . 28

IV General Conclusion 29

4 General Conclusion and Perspectives 30

4.1 General Conclusion 30
4.2 Answers to the Research Questions 31
4.3 Future Works and Perspectives 32

Bibliography **34**

List of Figures

2.1 Android development and release stages from source project to signed APK, based on Android build documentation [9, 11, 12]. 9
2.2 Main APK evidence regions used by static SBOM recovery, adapted from APK analysis documentation [13, 8]. 10
2.3 Android build and evidence-loss path from resolved dependencies to a recovered SBOM claim, adapted from Android build and APK analysis documentation [9, 13]. 12
2.4 Authority chain linking APK evidence to vulnerability interpretation. . . . 13

List of Tables

3.1 Technique taxonomy for Android component recovery. 17
3.2 Evidence-source taxonomy for reconstructed Android component claims. . 18
3.3 Transformation-threat taxonomy for Android component recovery. 20
3.4 Evaluation-practice taxonomy in the Android TPL literature. 23
3.5 Tier A studies classified by recovery role and trust implication. 24

4.1 Answers to the supporting research questions. 31
4.2 Priority perspectives for future Android APK-to-SBOM research. 32

List of Acronyms

Acronym	Definition
APK	Android Package Kit
API	Application Programming Interface
CVE	Common Vulnerabilities and Exposures
DEX	Dalvik Executable
ML	Machine Learning
NVD	National Vulnerability Database
OSV	Open Source Vulnerabilities
PURL	Package URL
R8	Android code shrinker and optimizer
SBOM	Software Bill of Materials
SCA	Software Composition Analysis
SPDX	Software Package Data Exchange
TPL	Third-Party Library

Part I

General Introduction

Chapter 1

General Introduction

1.1 Context

Software supply chains depend on large ecosystems of reusable components. SBOMs improve transparency by recording these components, but many audit scenarios provide only compiled software artifacts. In such cases, an SBOM is reconstructed from indirect and transformed evidence rather than generated from authoritative dependency metadata.

The Android ecosystem makes this problem especially visible. Applications are distributed as APK files after Gradle dependency resolution, D8 conversion, resource processing, signing, and often R8 shrinking or obfuscation. The final artifact may contain code from direct and transitive libraries, generated classes, bundled resources, native objects, and optimized fragments of dependencies whose original archive boundaries have disappeared. A build-time SBOM can observe declared coordinates before these transformations, while a binary-only recovery process must infer component identity and version from what remains.

This context links software composition analysis to vulnerability management. Public vulnerability records usually describe affected packages and version ranges. If the recovered coordinate is wrong, the advisory lookup is misdirected. If the recovered version is wrong or unsupported, an apparently precise CVE result can become a false alarm or a missed exposure. Android SBOM recovery must therefore be studied not only as a pattern-matching task, but also as an evidence-quality problem involving provenance, uncertainty, reference coverage, and transformation resilience.

1.2 Problem Statement

A recovered component name or version is not trustworthy merely because a tool emitted it. Trust depends on the origin, independence, transformation resilience, version specificity, and validation status of its evidence. Existing Android detection research provides strong technical signals, but lacks a unified model connecting those signals to claim provenance, uncertainty, and permitted vulnerability interpretation.

The practical difficulty is that Android recovery tools often output a single best candidate even when the supporting evidence is thin, transformed, or only partially representative of the shipped artifact. That habit is acceptable for exploratory screening, but it becomes fragile when the output is later used to justify a security decision. A name-only match

can be useful as a hint, yet the same match can be misleading if the version boundary is unclear, the reference corpus is incomplete, or the evidence was altered by shrinking or repackaging. The thesis therefore treats the gap between detector output and auditable claim as the real problem, not merely the gap between source code and binary code.

More precisely, the challenge is to define when a reconstructed claim is strong enough to support identity, when it is only strong enough to support a bounded version set, and when it should be reported as uncertain or withheld. Without that discipline, a binary recovery workflow can quietly turn evidence quality into false precision, especially in vulnerability contexts where a small version error can change the affected advisory set.

1.3 Motivation

Software transparency is most useful when an inventory can be inspected, challenged, and updated. Build-generated dependency metadata can often provide direct package coordinates and resolved versions, but auditors, incident responders, application stores, and downstream consumers may receive only a compiled artifact. In that setting, reconstructing an SBOM is necessary, yet the result is an inference from residual binary evidence rather than a direct export of the original build graph.

This distinction matters for vulnerability management. A package name may be sufficient to guide an investigation while remaining insufficient for an exact-version or affected-range decision. Conversely, an exact-looking version can be unsafe when the true release is absent from the reference base, when adjacent releases are feature-equivalent, or when optimization has removed the few discriminating features. If these conditions are hidden, a syntactically valid SBOM can encourage unsupported remediation priorities or false assurance.

The research reviewed in this thesis provides many useful signals for component recovery, including names, class and method structure, opcodes, literals, dependency relations, graphs, metadata, and learned representations. However, technical evidence and evaluation results answer different questions. Library identity, exact-version attribution, reference coverage, matcher recall, source validation, and vulnerability authority must not be collapsed into one confidence value or success rate.

The academic motivation is therefore to formalize how recovered evidence becomes an auditable claim. In this thesis, a reconstructed claim is *trustworthy* when its identity and version strength, supporting and opposing evidence, provenance, transformation exposure, unresolved uncertainty, validation context, and permitted downstream use are visible and mutually consistent. This definition allows partial knowledge and abstention to remain useful outcomes rather than forcing every analysis into an unsupported exact answer.

1.4 Research Questions

The central research question is:

> *How can heterogeneous evidence extracted from compiled and obfuscated Android applications be combined to produce trustworthy SBOM component and version claims with explicit provenance, confidence, uncertainty, and validation boundaries?*

This question is developed through six supporting questions:

1. Which evidence sources are available for recovering software components from compiled Android applications?
2. How have existing Android library-detection and binary component-recovery approaches combined these sources?
3. Which factors determine evidence strength for identity and exact-version attribution?
4. How should conflicting, missing, transformed, or source-dependent evidence be represented?
5. What information must accompany a reconstructed SBOM claim for it to be auditable and useful in vulnerability management?
6. How can such a framework be evaluated without overstating accuracy?

1.5 Objectives

The objective is to connect Android binary component-recovery techniques to a clear model of claim trust. The thesis reviews and classifies prior work, identifies the conditions that strengthen or weaken identity and version evidence, and derives directions for further research on trustworthy APK-to-SBOM reconstruction.

The resulting research objectives are:

1. a recovery-oriented taxonomy of techniques, evidence sources, transformation threats, and evaluation practices;
2. operational trust requirements based on provenance, evidence visibility, identity/version separation, uncertainty, validation context, vulnerability authority, and explicit denominators;
3. a synthesis of how recovered-claim models should distinguish identity, version, relationships, boundary observations, and downstream interpretation;
4. a synthesis of evidence hierarchy, fusion, conflict handling, confidence caps, bounded ambiguity, and abstention; and
5. a research agenda for broader calibration, validation, interoperability, and component domains.

These are research-synthesis objectives. The thesis does not present an implemented detector or a separate proposed system; it studies the evidence conditions that any future Android APK-to-SBOM recovery method should expose.

1.6 Scope

The primary domain is static reconstruction of software-component claims from compiled Android APKs. The analysis focuses on DEX-observable JVM/Maven components,

versioned reference artifacts, and the implications of recovered identity and version claims for SBOM production and vulnerability management. Android is used as a demanding case because compilation, shrinking, optimization, obfuscation, multidex packaging, artifact variants, and incomplete references can all change the evidence available after build time.

The thesis covers:

- structured review and taxonomy construction for Android third-party library and binary component-recovery research;
- evidence quality, provenance, transformation exposure, ambiguity, and validation boundaries for recovered claims;
- claim classes for exact, bounded, identity-only, report-only, abstained, and boundary outcomes;
- constraints on component-to-vulnerability interpretation; and
- research gaps that prevent detector outputs from becoming auditable SBOM claims.

The thesis does not propose a second APK detector, repeat a complete system implementation description, or evaluate a new tool. It does not claim universal probabilistic calibration, complete dependency recovery, or general native package/version attribution. Native shared objects, runtime-loaded code, encrypted payloads, commercial protectors, and other unsupported component domains are represented as boundary categories and future-research problems.

1.7 Thesis Organization

The remainder of the thesis is organized as follows.

Chapter 2 – Background and Research Methodology establishes the conceptual foundation. It distinguishes authoritative and reconstructed SBOMs, defines evidence, provenance, uncertainty, and software-component attribution, and describes the Android compiled-artifact context.

Chapter 3 – State of the Art and Taxonomy summarizes the frozen review protocol and applies it to the selected literature. It constructs taxonomies of recovery techniques, evidence sources, transformation threats, and evaluation practices, compares representative approaches without treating incompatible reported scores as one leaderboard, and derives the research gap addressed by the thesis.

Chapter 4 – General Conclusion and Perspectives answers the research questions, summarizes the review's main synthesis, and develops future directions including empirical calibration, independent validation, provenance-aware exchange, native and dynamic attribution, and human-centered evaluation.

Part II

Background

Chapter 2

Background

2.1 Introduction

This chapter establishes the concepts and review method used to reason about trustworthy recovery of software components from compiled Android applications. The thesis treats a recovered SBOM as a set of evidence-backed claims rather than as a simple list of names and versions. This distinction requires a common vocabulary for component identity, provenance, uncertainty, and transformation risk, followed by a transparent method for selecting and comparing prior research.

2.2 Software Supply Chains and SBOMs

Modern applications assemble code, resources, native binaries, generated artifacts, and transitive dependencies from many independently maintained projects. An SBOM records these components so that operators can reason about inventory, licensing, provenance, and known vulnerabilities. Standards define how such information is represented, but a representation format does not establish that the recorded component is actually present or that its exact version is known. This distinction is consistent with supply-chain guidance that treats component transparency as one control within a broader risk management process, and with empirical SBOM research showing persistent generation, consistency, and consumption challenges [1, 2, 3].

This thesis considers both SPDX and CycloneDX, while using Package URL as a standard package-identity model [4, 5, 6]. These standards therefore define the target vocabulary and exchange structures, while the literature synthesis defines the evidential conditions under which individual fields may be populated responsibly.

2.2.1 Authoritative and Reconstructed SBOMs

An authoritative build-time SBOM can use dependency resolution metadata, lockfiles, build graphs, and package-manager records. By contrast, an artifact-reconstructed SBOM begins after those authoritative records have been removed or made unavailable. It must infer a component from observable residue in an APK, compare that residue with references, and preserve the uncertainty introduced by compilation and incomplete coverage.

The distinction is central to this thesis. A reconstructed component identity may be credible even when its exact version is unresolved. Similarly, a syntactically valid CycloneDX or SPDX document may contain semantically weak claims. Trustworthiness is consequently a property of each claim and its evidence trail, not of the file format alone.

2.3 Android Applications and Build Cycle

Android applications are built from source code, framework components, resources, and packaged dependencies. The platform exposes a clear execution model to the developer, but that model is partly transformed before the application reaches an analyst. Understanding the development cycle is therefore essential before discussing binary recovery.

2.3.1 Application Components

Android applications are organized around a small set of component types: activities for user-facing screens, services for background work, broadcast receivers for event-driven actions, and content providers for structured data sharing [7]. These components are declared in the manifest, linked to permissions and intent filters, and connected to the Android framework through runtime callbacks.

The manifest is important because it records the application package name, component declarations, permissions, SDK constraints, and other configuration data [8]. However, it does not by itself represent the full dependency graph of the application. A manifest can therefore support contextual interpretation, but it cannot be treated as a complete SBOM.

2.3.2 Development, Build, Packaging, and Release

Android development typically uses Gradle and the Android Gradle Plugin to resolve source modules and external libraries before compilation [9, 10]. A dependency may resolve to different physical artifacts depending on platform, build type, flavor, or metadata-only packaging. The build cycle usually proceeds through source compilation, resource processing, bytecode conversion, packaging, signing, and distribution.

Java and Kotlin source code is first compiled to JVM bytecode. D8 then converts that bytecode into DEX form for the Android runtime, while R8 may shrink, optimize, and obfuscate the release build [11, 12]. The final artifact therefore reflects both the build configuration and the optimization policy, not only the original source tree.

In this context, the *source project* denotes the developer-visible workspace: source files, Gradle scripts, manifests, resources, and declared dependencies. The *dependency graph* is the resolved set of direct and transitive libraries selected for a concrete build variant. A build variant is important because debug and release builds, Android API targets, flavors, and platform attributes can lead to different resolved artifacts. The build graph is therefore stronger evidence than the final APK for declaring an SBOM, but it is not always available to an external analyst.

Resolved libraries usually arrive as JAR or AAR artifacts. A JAR mainly contains JVM bytecode and metadata, whereas an AAR is Android-specific and may also contain resources, a manifest fragment, assets, and native libraries. During packaging, these artifacts are merged with the application code. Compilation and conversion then change

their representation: D8 emits Android DEX bytecode, and R8 can remove unused classes, inline methods, rename identifiers, merge classes, and rewrite call sites. *Shrinking* removes unused code and resources, while *obfuscation* makes names less recognizable. The signed APK is the distribution artifact after these transformations and after integrity/authenticity metadata has been added.

Figure 2.1 summarizes this development and release path. The figure is important because it separates the places where an ideal build-time SBOM can be produced from the later artifact that binary recovery must inspect.

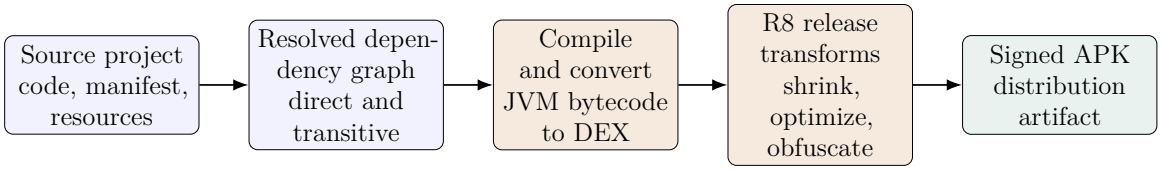


Figure 2.1: Android development and release stages from source project to signed APK, based on Android build documentation [9, 11, 12].

2.3.3 APK Structure and DEX

An APK is a ZIP-format archive that usually contains one or more DEX files, the manifest, compiled resources, raw assets, native libraries, and signing material [13]. The DEX files contain types, strings, fields, methods, classes, annotations, and instructions. When an application exceeds the single-DEX reference limit, multidex packaging introduces files such as `classes.dex` and `classes2.dex` [14].

This structure matters for recovery because each region contributes different evidence. The manifest provides configuration context, DEX files expose code and structural relationships, resources may reveal package or product hints, assets may contain bundled metadata, and native libraries may reveal additional files that are relevant to provenance or vulnerability analysis. None of these regions is a complete inventory by itself.

Figure 2.2 shows the APK as a set of complementary evidence regions. DEX is the main executable evidence because it contains the classes, methods, strings, fields, and instructions that remain after compilation. The manifest is declarative evidence: it names components, permissions, SDK constraints, package identity, and other build-time decisions. Resources and assets are packaging evidence; they may preserve names, configuration files, or markers from libraries even when Java/Kotlin names have been changed. Native files, typically `.so` libraries, are binary objects that may have their own provenance and vulnerability surface. Signing metadata authenticates the APK package but does not prove the correctness of any recovered component claim.

Two additional concepts are important for structured evidence mapping. First, *references* denote external method, field, or type calls within the DEX bytecode that target external packages or standard Android libraries. These references survive renaming and shrinking because R8 must preserve call site targets to maintain functionality. Second, *build metadata* includes declared SDK levels, version codes, build variant configurations, and signing material. While this metadata is easy to modify, it provides essential context for verifying the integrity and timeline of the analyzed APK.

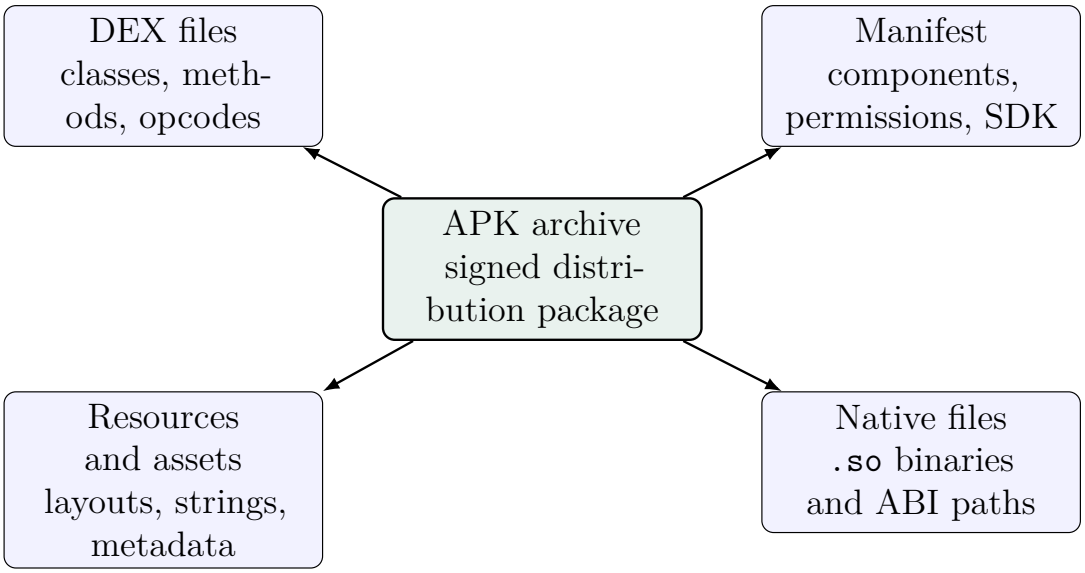


Figure 2.2: Main APK evidence regions used by static SBOM recovery, adapted from APK analysis documentation [13, 8].

2.3.4 Libraries and Dependencies

Android applications reuse both direct and transitive dependencies. A direct dependency is declared explicitly in the build; a transitive dependency is pulled in by another dependency. Variant-aware resolution can select Android, JVM, Kotlin, or metadata artifacts for the same coordinate [10]. That is why the binary artifact may differ from the original source dependency declaration even when the build is correct.

Libraries are commonly distributed as JAR or AAR files before compilation [15]. Their coordinates are usually expressed in Maven form, which makes them useful for SBOM generation and vulnerability queries. However, once those artifacts have been merged, transformed, or shaded into an APK, the library boundary is no longer directly visible. Binary recovery must therefore infer whether an observed feature belongs to a library family, a specific version, or only an uncertain candidate set.

2.3.5 Static and Dynamic Analysis

Static analysis inspects the APK without executing it, using archive parsing, manifest decoding, resource inspection, DEX disassembly, and structural feature extraction [16]. Dynamic analysis observes the application while it runs, which can reveal runtime-loaded code or behavior that is invisible in the static archive [17]. Both approaches are useful, but they answer different questions.

This thesis focuses on static analysis because the goal is to recover evidence from the delivered artifact itself. Dynamic analysis remains relevant as a future extension or validation aid, especially for encrypted payloads, custom class loaders, and other runtime-only behaviors [18].

2.4 Trust, Evidence, Provenance, and Uncertainty

2.4.1 Evidence and Claims

An evidence item is an observable fact used to support or challenge a component claim. Examples include a package fragment, an opcode fingerprint, a class-dependency relation, a resource marker, a native-library name, or a source-validated expected dependency. A claim is the interpretation built from one or more such items, for example that a library family is present or that a particular version is the best-supported candidate.

In this context, a *bounded claim* refers to a recovered SBOM assertion whose certainty and completeness are strictly limited by the available binary evidence and compiler transformations. Unlike a build-time SBOM, which makes authoritative assertions based on complete project declarations, a recovered SBOM can make only partial claims that are bounded by what survived shrinking, obfuscation, and method inlining.

2.4.2 Provenance and Independence

Provenance records where evidence came from, how it was extracted, and which transformations may have affected it. Two matching fields are not independent when they are derived from the same underlying artifact feature. Confidence must therefore not be increased merely by counting correlated signals as separate confirmations. The framework adopts the W3C PROV distinction between entities, activities, and agents as a general provenance vocabulary, while specializing it for APK observations, reference profiles, validation artifacts, and analysis steps [19].

2.4.3 Uncertainty and Abstention

Uncertainty can arise from missing evidence, conflicting evidence, incomplete reference coverage, adjacent versions with near-identical code, partial library inclusion, or destructive optimization. The framework allows ambiguity sets and abstention because an explicit unresolved result is more auditable than a forced exact version. Reject-option research provides the general decision-theoretic precedent for withholding a classification when the expected error cost is too high [20]. This policy also limits downstream vulnerability authority: a vulnerability finding cannot be stronger than the identity and version evidence on which it depends.

2.5 Android Compiled-Artifact Context

The build-cycle concepts above explain why compiled Android evidence is structurally weaker than source-level dependency data. Android compilation and R8 transformations directly affect the stability and interpretability of recovered evidence [12].

Java or Kotlin source is compiled and transformed into DEX bytecode, while resources and manifests are packaged into an APK. Android applications therefore carry both executable code and declarative metadata: the final artifact can still expose components, permissions, SDK constraints, resource names, and native binaries, but those signals no longer appear in the same shape as the original source tree [7, 9, 8, 13]. Dependency

boundaries that were explicit during the build may disappear when classes are merged into one or more DEX files. R8 can rename identifiers, remove unused code, rewrite control flow, merge classes, inline methods, and change call sites. Packaging may also introduce multidex layouts, shaded or relocated code, native libraries, generated classes, and dynamically loaded components.

These transformations do not affect all evidence equally. Names and package trees are inexpensive but fragile. Opcode, control-flow, call, and dependency structures can survive renaming, yet they are still altered by shrinking and optimization. Resources and metadata can provide independent support, but they may identify only a family rather than an exact version. The state-of-the-art taxonomy in chapter 3 therefore classifies both evidence sources and the transformations that threaten them.

Figure 2.3 illustrates why binary recovery is weaker than build-time inventory generation. The dependency graph is visible before compilation, but the audit target is often the transformed APK after boundaries, names, and unused code have changed.

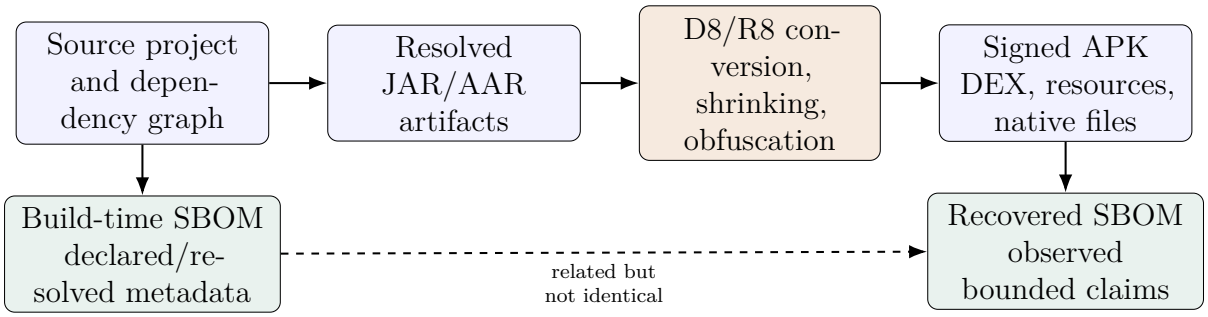


Figure 2.3: Android build and evidence-loss path from resolved dependencies to a recovered SBOM claim, adapted from Android build and APK analysis documentation [9, 13].

2.6 Software Component Attribution

2.6.1 Identity, Version, and Security Authority

Component attribution contains at least three distinct decisions:

1. **identity attribution**: whether a component family or coordinate is present;
2. **version attribution**: which exact version or bounded version set is supported; and
3. **security interpretation**: which vulnerability statements are permitted by the preceding evidence.

A method that performs well at library-family detection does not automatically provide exact-version authority. Likewise, a vulnerability database match is not confirmation that the affected version is present. Keeping these decisions separate prevents a plausible identity match from being silently promoted into an exact and security-critical statement.

Figure 2.4 shows the authority chain. Each step depends on the previous one: vulnerability interpretation is meaningful only when component identity and version evidence are strong enough for the affected-range query being made.

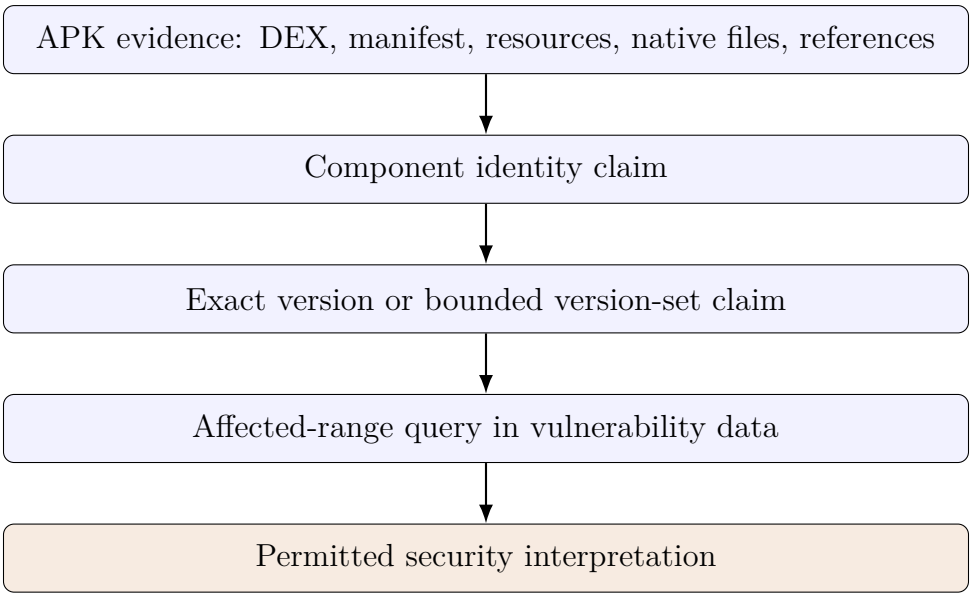


Figure 2.4: Authority chain linking APK evidence to vulnerability interpretation.

2.6.2 Reference Dependence

Many attribution methods compare an application with profiles extracted from known library releases. Such a method can recognize only what its reference corpus represents. Failure may therefore result from the matcher, the extractor, or a missing reference rather than from absence of the component. Reference coverage is treated as a first-class validity condition throughout the thesis.

2.7 Evidence Sources in APKs

The review extracts evidence into recurring families:

- lexical and namespace evidence, including package and class names;
- bytecode and semantic evidence, including opcodes, API references, literals, and method descriptors;
- structural evidence, including control-flow, call, inheritance, and class-dependency graphs;
- resource and packaging evidence, including manifests, resource names, assets, and native-library files;
- external reference and metadata evidence, including library profiles, Maven coordinates, dependency relations, and release chronology;
- learned representations derived from bytecode or structural features;
- validation evidence, including controlled builds, source manifests, and source-based observations.

No family is assumed to be universally authoritative. The framework evaluates its origin, transformation resilience, version specificity, independence, coverage, and validation status.

2.8 Conclusion

This chapter separated a valid SBOM representation from the evidential strength of its contents and defined component attribution as a sequence of identity, version, and security-authority decisions. It also froze a structured mapping protocol that prevents unavailable full texts, incompatible metrics, and correlated signals from being silently promoted into strong claims.

These background concepts explain why Android APK-to-SBOM recovery cannot be evaluated through a single accuracy value. The same output may be trustworthy as a library-family hint, weak as an exact-version assertion, and unsafe as a vulnerability conclusion. A reliable review must therefore preserve the evidence source, transformation setting, reference coverage, denominator, and validation context attached to each reported result. The next chapter summarizes the frozen review protocol and applies these rules to classify the state of the art and identify the remaining research gaps.

Part III

State of the Art

Chapter 3

State of the Art and Taxonomy

3.1 Introduction

This chapter applies the structured mapping protocol defined in chapter 2 to the Android third-party library detection literature. The frozen review protocol follows software-engineering review guidance [21] and uses a fixed search scope, screening logic, extraction schema, and quality assessment so that the corpus can be discussed transparently even when the underlying studies use different datasets, task units, obfuscation settings, and ground-truth procedures. Instead of producing a numerical leaderboard, the chapter organizes prior work by technique, evidence source, transformation threat, and evaluation practice.

The analysis is centered on the 15 Tier A studies whose full texts are available locally. Tier B records are useful for search coverage and for identifying emerging directions, especially SBOM-oriented recovery and recent benchmarking, but they are not used here as independent authority for precise quantitative claims. This boundary keeps the review consistent with the protocol and prevents unverified metadata from receiving the same weight as inspected studies. Fields not reported by a Tier A publication are treated as not reported rather than inferred, and original-paper claims are kept separate from later cross-tool observations.

The chapter uses the term *component recovery* broadly. A study may recover a library family, a package cluster, a class set, an exact version, a vulnerability-related version list, or only a presence signal. For trustworthy SBOM reconstruction, these outputs are not equivalent. A method that detects the presence of a library family does not necessarily justify an exact Maven version, and an exact-version candidate does not automatically justify a CVE claim unless the evidence, reference coverage, and validation context are visible.

The review protocol is therefore part of the interpretation of the chapter. Searches combined Android artifacts, third-party components or SBOM terms, detection or attribution tasks, and transformation-risk keywords; records were deduplicated and screened before being assigned to evidence tiers; and each included study was extracted across task granularity, inputs, evidence families, transformation coverage, evaluation setting, and stated limitations. Studies without locally verified full text were treated as supplementary records rather than as independent authority, which keeps the chapter's synthesis aligned with the available evidence.

3.2 Taxonomy of Component-Recovery Techniques

The reviewed approaches can be grouped into seven recurring technique families. The categories are not mutually exclusive: most mature tools combine several families, for example by using a fast coarse filter followed by a slower structural match. The taxonomy therefore describes the role each family plays in a recovered component claim.

Table 3.1: Technique taxonomy for Android component recovery.

Technique family	Typical evidence	Trust role	Tier A examples
Namespace and metadata heuristics	Package trees, class names, library naming conventions	Cheap candidate generation, but weak authority when names are renamed, flattened, relocated, or shaded.	Assessed in [22, 23]
Profile, hash, and signature matching	Method descriptors, Merkle trees, fuzzy hashes, class or method signatures	Can support identity and, when references are complete, version claims. Its authority is bounded by reference coverage and by the stability of the hashed features under optimization.	**LibScout** [24], **ORLIS** [25], **LibID** [26]
Structural graph segmentation	Call, inheritance, containment, and class-dependency relations	Improves module boundary recovery when package structure is unreliable. Supports stronger identity evidence, but does not by itself solve exact version attribution.	**LibD** [27], **LibPecker** [28], assessment studies [22, 23]
Coarse-to-fine matching	Fast filtering followed by detailed opcode, CFG, or structural comparison	Balances scalability and precision. Trust depends on both stages: a missed coarse candidate can cap recall, while an over-permissive fine stage can inflate false positives.	**ORLIS** [25], **ATVHUNTER** [29], **LibScan** [30]
Version-discriminating semantic features	Opcode sequences, basic-block features, call chains, field-operation patterns, literals, and class summaries	Provides the main route from library identity to exact-version authority. Adjacent versions and optimized builds remain difficult because many versions share most code.	**ATVHUNTER** [29], **LibScan** [30], **LibHunter** [31], **SAD** [32]

Technique family	Typical evidence	Trust role	Tier A examples
Learned representations	Smali token sequences, structural vocabularies, neural embeddings	Can learn obfuscation-resilient presence signals, but the reviewed Tier A learning approach does not provide exact-version or boundary authority on its own.	**LibAttention** [33]
Empirical assessment and benchmarking	Shared datasets, patched tools, source-known applications, evaluation criteria	Does not directly recover components, but defines which claims are comparable and exposes tool failure modes, missing denominators, and reproducibility limits.	**Tool assessments** [22, 23], **Ramirez thesis** [34]

The movement across these families is historically important. Early methods made library profiles and package structure usable at scale, but later studies increasingly treat package names as unreliable evidence. Graph-based and version-aware methods improve the situation by using relations, opcode content, and class behavior. The newest verified Tier A work adds learned representations and finer class-level summaries. Even so, the taxonomy shows that standard SBOM emission remains outside the main output model of the verified Android TPL literature: most studies output names, clusters, versions, class mappings, or vulnerability-linked lists rather than a provenance-rich CycloneDX or SPDX document.

3.3 Taxonomy of Evidence Sources

An evidence source is useful for trustworthy SBOM recovery only when its origin, stability, version specificity, and provenance are understood. The same signal may be strong for identity and weak for exact version. Conversely, a highly version-specific literal may be too fragile to survive resource removal or string encryption. Table 3.2 summarizes the recurring evidence families extracted from the Tier A studies.

Table 3.2: Evidence-source taxonomy for reconstructed Android component claims.

Evidence family	Examples	Typical stability	Trust implication
Names and namespaces	Package prefixes, class names, manifest names	Weak under renaming, package flattening, relocation, and shading	Useful for fast hints and human inspection, but insufficient for exact SBOM claims without corroboration.

Evidence family	Examples	Typical stability	Trust implication
Descriptors and API surfaces	Method signatures, fuzzy type descriptors, Android API usage	More stable than names under identifier renaming; weaker under dead-code removal and inlining	Supports identity and coarse version discrimination when combined with reference profiles.
Opcode and instruction sequences	Basic-block opcodes, method opcode chains, call-chain opcodes	Resilient to naming changes; threatened by optimization, inlining, and control-flow rewriting	Provides version-discriminating evidence, especially when compared against known library releases.
Control-flow and dependency structures	CFGs, call graphs, inheritance graphs, class-dependency graphs	Stable against simple renaming and often against package restructuring; affected by optimization and partial inclusion	Improves module boundary recovery and reduces dependence on package layout.
Literals, constants, and resources	Strings, constants, resource identifiers, assets	Potentially version-specific, but fragile under stripping, string encryption, and resource rewriting	Best used as tie-breakers or independent corroboration, not as sole authority.
Reference corpus and release metadata	Library profiles, Maven coordinates, release chronology, known vulnerable version ranges	External to the APK; stable only if the corpus is complete and correctly constructed	Defines the ceiling for recognizable identities and versions. Missing references must not be interpreted as component absence.
Learned representations	Transformer embeddings over Smali tokens, structural vocabularies	Can absorb complex obfuscation patterns, but may be hard to audit feature by feature	Promising for presence screening and robustness, but provenance and exact version authority require additional explanation.
Validation evidence	Controlled builds, source manifests, F-Droid dependency data, source-known ground truth	Strong within the validation context; not automatically generalizable	Can upgrade a claim's evaluation status, but must remain separate from APK-only inference.

The trust implication of this taxonomy is that evidence fusion should prefer independent and complementary evidence over repeated variants of the same feature. For example, several hash matches derived from the same method body are less independent than a method-level match corroborated by a class-dependency relation and a version-specific literal. This distinction supports the thesis synthesis: independence, provenance, and reference coverage must be treated as first-class quality dimensions whenever detector

outputs are interpreted as SBOM claims.

3.4 Taxonomy of Transformation Threats

Android component recovery is difficult because compilation and packaging can remove the very boundaries that build-time dependency tools rely on. R8 and related obfuscators may rename identifiers, flatten packages, remove unused code, rewrite control flow, and inline methods [12]. The reviewed literature shows that no single evidence family remains stable under all transformations.

Table 3.3: Transformation-threat taxonomy for Android component recovery.

Threat	Evidence most affected	More resilient evidence	Trust consequence
Identifier renaming	Class, method, field, and package names	Fuzzy descriptors, opcode features, structural relations	Name matches cannot be treated as independent authority unless corroborated.
Package flattening and relocation	Package-subtree segmentation and namespace heuristics	Class-dependency graphs, call relations, profile matching independent of package roots	Package-based module boundaries become unreliable; graph segmentation becomes more important.
Control-flow rewriting	CFG shape, branch structure, fuzzy CFG hashes	Higher-level dependency relations, method descriptors, class summaries	CFG-only methods may overstate robustness when control-flow transformations are not in the evaluation.
Dead-code removal and shrinking	Presence of optional classes, methods, API calls, literals	Partial-match scoring, class-level confidence, source-known validation	Absence of a feature may indicate transformation rather than absence of the library.
Method inlining and outlining	Method boundaries, call-chain evidence, version-specific method bodies	Multi-scale evidence, class-level summaries, literals, residual dependency structure	Exact-version authority must be capped when inlining changes the features used for version discrimination.
Multidex and split packaging	Single-file DEX assumptions and incomplete scanners	Whole-APK extraction over all DEX files and split artifacts	Evaluation denominators must state whether all executable artifacts were analyzed.
Partial library inclusion	Binary present/absent decisions and complete-profile matching	Similarity scores, per-class evidence, explicit ambiguity classes	A partial match should not automatically become a full component/version claim.

Threat	Evidence most affected	More resilient evidence	Trust consequence
Nested or transitive dependencies	Flat library lists and one-library-one-package assumptions	Dependency graphs, source-assisted validation, relationship-aware SBOM models	A recovered component may be direct, transitive, bundled, shaded, or copied; the relationship must remain uncertain unless evidenced.
Native and dynamic code	DEX-only static features	Native file hashes, load-site evidence, dynamic analysis, source validation	Out-of-scope code must be represented as a boundary, not silently ignored.

The most consequential threats for exact-version claims are shrinking, partial inclusion, and method inlining. Identifier renaming is comparatively well handled by modern structural or opcode-based approaches, but version evidence often depends on small differences between adjacent releases. Optimizations that delete, merge, or inline exactly those differences can turn an apparently confident match into a version-band ambiguity.

3.5 Representative Android TPL Approaches

3.5.1 From Profile Matching to Structural Boundaries

LibScout established a reference-profile model for matching compiled Android applications against known library releases. Its three-layer profile structure and fuzzy method descriptors made identifier renaming less damaging, and its similarity scores provided a graded alternative to exact profile equality [24]. From the viewpoint of trustworthy SBOM recovery, the important lesson is dual: reference profiles can support version claims, but those claims inherit the assumptions of the profile structure and the reference corpus. When package layout or method boundaries are transformed, the confidence of the resulting claim must be reduced rather than silently preserved.

LibD moved away from pure package-layout dependence by recovering library instances through a homogeny graph over inclusion, inheritance, and call relations, combined with opcode-derived fingerprints [27]. This made large-scale library discovery more resilient to naming changes, but the output remained a clustering-oriented library identification result rather than an exact-version or SBOM result. For the taxonomy, LibD is therefore strong evidence for graph-based boundary recovery and weak evidence for version authority.

ORLIS introduced a two-stage obfuscation-resilient design. Its first stage uses inter-procedural call-graph-derived features and `sdhash` to remove unlikely library candidates. Its second stage applies `ssdeep` class signatures and an injective class mapping to verify whether a library is included in the application [25]. The design is valuable because it separates scalable filtering from detailed confirmation and can represent partial inclusion without relying on package names. Its limitations are equally important for the proposed framework: a class mapping is not automatically an exact-version claim, and computational cost affects whether a method can be used as routine SBOM evidence.

LibPecker and LibID further illustrate the shift from names to structural and class-

level evidence. LibPecker emphasizes class dependency and adaptive similarity. LibID formulates consistent class correspondence with binary integer programming and identifies specific library versions under code shrinking and package modification [28, 26]. Together with the later assessment studies, they support the review's central classification: package structure is a fragile feature family, while class relationships, signatures, and opcode-derived evidence are more suitable as identity support under obfuscation.

3.5.2 Version-Aware and Vulnerability-Oriented Pipelines

ATVHUNTER combines class-dependency based decoupling, coarse CFG filtering, fine-grained opcode matching, and a vulnerability database to identify vulnerable library versions [29]. It is one of the most important verified studies for this thesis because it connects version identification to security interpretation. However, its output remains a version and vulnerability list rather than a standard SBOM with explicit provenance, uncertainty, and claim boundaries. The study therefore motivates the framework's rule that vulnerability authority cannot exceed the strength of the underlying version evidence.

LibScan and LibHunter extend the version-aware family by using richer opcode, class-correspondence, call-chain, and optimization-aware evidence [30, 31]. Their relevance is not only that they improve matching. They also show that code optimization itself is an evaluation dimension. In particular, LibHunter augments reference features to model call-site optimization and performs cross-inlining method matching. A method evaluated only on clear or lightly obfuscated applications has not demonstrated the same authority as a method evaluated under inlining and shrinking.

SAD provides the most detailed Tier A example of class-structural version-level reasoning. It splits classes into stateful and stateless categories, applies field-operation sequences or opcode similarity accordingly, and uses literals to break ties between adjacent versions [32]. This design supports the thesis claim that exact-version attribution requires evidence at a finer granularity than library presence. At the same time, its remaining sensitivity to method inlining confirms that strong reported version-level performance does not eliminate transformation risk.

3.5.3 Learning-Based Presence Detection

LibAttention represents a different family: instead of comparing explicit profiles, it learns representations from large Smali corpora and predicts library presence with a Transformer-based model [33]. This is important because learned features can capture patterns that rule-based signatures miss under aggressive obfuscation. For SBOM reconstruction, however, the reviewed Tier A result is bounded: it predicts presence rather than exact version, does not expose an explicit class or library-boundary mapping, and does not provide standard SBOM output. In the taxonomy, learned presence detection is therefore a potentially useful candidate-generation signal but not sufficient version or vulnerability authority by itself.

3.5.4 Empirical Assessments and Cross-Tool Studies

The ASE 2020 assessment and the later systematic assessment are not merely background surveys; they define several trust requirements for any recovered SBOM pipeline [22,

23]. They show that tool results depend on module decoupling, multidex handling, AAR processing, obfuscation setting, version-identification capability, efficiency, and ease of reproduction. These studies are a warning against reporting a single accuracy value without denominators, exclusions, and task levels.

The Ramirez thesis provides a cross-platform empirical comparison of Android and iOS library-detection tools [34]. Its Android study reuses 75 APKs from the ORLIS dataset, 75 from the LIBLOOM dataset, and 50 manually selected Google Play applications. It is therefore useful for cross-tool execution and dataset-reuse lessons, but it is not a source-known or F-Droid validation study and it does not independently establish standard SBOM output.

TPL-Benchmark strengthens the benchmark perspective with a large, version-annotated Android dataset and a standardized reevaluation of ten detection methods [35]. Its compatible tool-evaluation slice shows that the best library-level F1 score was 60.15%, while LibHunter obtained the highest reported version-level recall at 76.48%. These results are not a universal ceiling, but they demonstrate that realistic shared ground truth can produce substantially weaker and more heterogeneous outcomes than isolated original evaluations.

UniBOM is an adjacent rather than Android-specific study. It combines firmware extraction, filesystem scanning, source-code analysis, SBOM generation, and vulnerability visualization, and evaluates the workflow on 258 wireless gateway firmware images and four IoT operating-system codebases [36]. It confirms that binary, metadata, and source evidence can coexist in one SBOM workflow. It does not, however, solve APK-side Maven exact-version attribution or provide ground truth for Android TPL detection.

3.6 Evaluation Practices and Ground Truth

The reviewed studies evaluate different questions even when they use similar words such as detection, identification, precision, recall, or version accuracy. A trustworthy synthesis must therefore classify evaluation practice before interpreting the metric.

Table 3.4: Evaluation-practice taxonomy in the Android TPL literature.

Evaluation practice	What it can show	Boundary for SBOM trust
Controlled library or reference corpus	Whether a matcher can distinguish known library profiles or versions under specified transformations	Results depend on reference completeness and may overstate performance on diverse real applications.
Synthetic or tool-based obfuscation tests	Whether selected transformations damage a feature family	Only covers the transformations and configurations tested; R8 inlining and shrinking require explicit treatment.
Market-scale discovery	Whether a detector scales and finds plausible library families in many apps	Without source ground truth, discovery volume is not the same as correctness or exact-version authority.

Evaluation practice	What it can show	Boundary for SBOM trust
Open-source or source-known validation	Whether recovered outputs match dependency evidence available outside the APK	Strong for audited cases, but it is a validation lane, not proof that the same APK-only method is generally correct.
Cross-tool benchmark or assessment	Which tools fail under shared criteria and where methodology is inconsistent	Useful for gap analysis, but metrics remain bounded by the benchmark's dataset and task definitions.
Security or vulnerability mapping	Whether detected versions can be connected to vulnerability data	The vulnerability conclusion is valid only as far as the component identity, exact version, and vulnerability database mapping are valid.

This taxonomy explains why the chapter does not pool reported performance numbers. LibScout's profile precision, ORLIS's obfuscation precision, LibD's market-scale discovery volume, ATVHUNTER's version-level results, LibAttention's R8-resilient presence detection, SAD's class-structural version scores, and TPL-Benchmark's cross-tool results answer different questions [24, 27, 25, 29, 33, 32, 35]. For a reconstructed SBOM, the evaluation question must be restated as: *which claim type is being validated, against which denominator, and under which transformations?*

3.7 Comparative Analysis

Table 3.5 applies the chapter taxonomy to the Tier A seed corpus. The final column states the trust interpretation rather than a simple score. This is deliberate: the same study can be strong evidence for one claim type and weak evidence for another. The table records only supported roles and limitations; missing study fields are not filled by inference.

Table 3.5: Tier A studies classified by recovery role and trust implication.

Study	Primary role	Main evidence and technique	SBOM-trust implication
LibScout [24]	Reference-profile identity and version matching	Fuzzy method descriptors, package-class-method profile hierarchy, similarity score	Supports the idea of version profiles, but package and method-structure assumptions must be exposed as evidence limits.
LibD [27]	Scalable library discovery and boundary recovery	Homogeny graph, call/inheritance/inclusion relations, opcode-derived fingerprints	Strengthens identity evidence under name obfuscation, but does not provide exact-version or SBOM authority.

Study	Primary role	Main evidence and technique	SBOM-trust implication
LibPecker [28]	Class-dependency based detection	Class dependency hashing, adaptive similarity, weighted class matching	Shows that structural class relations are useful, while similarity still needs explicit thresholds and uncertainty handling.
ORLIS [25]	Obfuscation-resilient structural matching	Call-graph-derived coarse features with `sdhash`, followed by `ssdeep` class signatures and injective class mapping	Provides partial-inclusion evidence, but high cost and missing exact-version authority limit direct SBOM use.
LibID [26]	Obfuscated library and version identification	Consistent class correspondence through binary integer programming, with source-backed threshold evaluation	Supports specific-version claims under its tested transformations; downstream security authority still depends on reference and vulnerability evidence.
ASE 2020 assessment [22]	Cross-tool failure analysis	Unified criteria, tool patches, multidex and AAR observations	Defines required evaluation dimensions and warns that engineering blind spots can dominate measured accuracy.
Systematic assessment [23]	Broader comparative methodology	Six evaluation criteria, module-decoupling comparison, tool availability and limitations	Supports taxonomy-based synthesis and shows that CDG-style evidence is preferable to package-only evidence.
ATVHUNTER [29]	Version and vulnerability-oriented detection	CDG decoupling, CFG coarse filter, opcode fuzzy matching, vulnerability DB	Strongly connects version evidence to security use, but still lacks standard SBOM provenance and full R8-inlining coverage.
LibScan [30]	Precise TPL identification	Class correspondence, method opcode similarity, call-chain evidence	Improves fine-grained matching evidence; claim strength remains tied to reference coverage and transformation conditions.
Ramirez thesis [34]	Cross-platform tool comparison	ORLIS and LIBLOOM Android datasets, manually selected Google Play apps, and Android/iOS contrast	Supports cross-tool and dataset-reuse analysis, but does not provide source-known Android ground truth or standard SBOM validation.

Study	Primary role	Main evidence and technique	SBOM-trust implication
LibHunter [31]	Optimization-aware detection	Call-site-optimization feature enhancement and cross-inlining method matching	Makes optimization a first-class validity condition for exact-version claims.
LibAttention [33]	Learning-based presence detection	Transformer over Smali tokens and SDK structural vocabulary	Promising for obfuscation-resilient identity hints, but insufficient alone for exact versions, provenance, or CVE authority.
SAD [32]	Class-structural version-level detection	CDG neighbor aggregation, stateful/stateless split, field-operation sequence, literal tie-breakers	Provides strong version-discriminating evidence, while remaining sensitive to method inlining and lacking standard SBOM output.
TPL-Benchmark [35]	Ground-truth benchmark and cross-tool reassessment	Version-annotated real applications, dependency extraction, and standardized evaluation of ten detectors	Provides realistic comparative evidence and exposes low or uneven generalization; it diagnoses detector limits rather than supplying a detector.
UniBOM [36]	Adjacent SBOM generation and analysis workflow	Firmware extraction, filesystem and source scanning, SBOM generation, CVE analysis, and visualization	Shows end-to-end SBOM integration outside Android TPL attribution, but cannot justify APK Maven identity or exact-version claims.

Three patterns emerge from this comparison. First, the strongest exact-version approaches are multi-evidence approaches: they do not rely on names alone but combine structural, opcode, class, and literal information. Second, evaluation authority is not the same as detection authority. Assessment studies and source-known validation do not identify libraries by themselves, but they determine whether a result can be interpreted safely. Third, standard SBOM trust requirements are largely external to the detector output: provenance, ambiguity, validation class, relationship type, and vulnerability authority are rarely represented as first-class fields.

3.8 Limitations of Existing Research

The taxonomy exposes limitations that recur across the verified literature:

1. **Library identity and exact version are often conflated.** Presence detection, class-set matching, and exact-version attribution are different claims, but tool outputs do not always preserve the distinction.

2. **Reference coverage is rarely treated as part of the result.** A missed match can result from absent references, extraction failure, or a weak matcher. Without coverage metadata, the user cannot tell which failure occurred.

3. **Transformation coverage is incomplete.** Identifier renaming is widely considered, but shrinking, partial inclusion, package flattening, and method inlining remain unevenly evaluated [31, 33, 32].

4. **Confidence is usually a threshold outcome, not an auditable explanation.** Many methods compute similarities or classifier scores, but few expose why a claim should be trusted, which signals conflicted, or why the method did not abstain.

5. **Partial libraries and dependency relationships are not fully represented.** Similarity scores can detect partial inclusion, but a standard SBOM also needs relationship semantics: direct, transitive, bundled, shaded, copied, or uncertain.

6. **Vulnerability interpretation is stronger than the underlying evidence in many pipelines.** A detected library name is not enough for a version-specific CVE claim. Even a plausible version candidate requires reference, transformation, and validation context.

7. **Reproducibility remains uneven.** Tool availability, installation difficulty, dataset access, and benchmark construction vary across studies, limiting independent confirmation of reported results.

8. **Standard SBOM output is not the dominant research artifact.** The verified studies mostly produce detector outputs rather than complete CycloneDX or SPDX statements with evidence, provenance, and uncertainty fields.

These limitations do not make the existing work weak. Rather, they show that the literature has mostly optimized detection performance, while trustworthy SBOM recovery requires an additional layer that explains what each recovered claim means and how it may be used.

3.9 Research Gap

The state of the art provides valuable techniques for recognizing Android third-party libraries from compiled artifacts. Reference profiles, fuzzy descriptors, dependency graphs, opcode features, call-chain evidence, stateful/stateless class summaries, learned Smali representations, and source-known validation all contribute useful evidence. Within the reviewed Android TPL literature and adjacent binary-SBOM studies, however, no verified study provides a unified model that connects these signals to a reconstructed SBOM claim while preserving all of the trust dimensions used in this thesis.

The research gap can therefore be stated as follows:

> Existing Android component detectors provide useful identification and version-discrimination techniques, but they do not consistently attach each recovered SBOM claim to its evidence provenance, reference coverage, transformation risk, confidence boundary, ambiguity, validation class, and permitted vulnerability interpretation.

This gap is not only a formatting problem. A syntactically valid SBOM can still be untrustworthy if it hides uncertainty, promotes identity evidence into exact-version authority, or turns an ambiguous version set into a definitive CVE finding. Conversely, a detector that abstains or reports a bounded version set may be more useful for security decision-making than a pipeline that always emits a single version without explaining the evidence.

3.10 Conclusion

This chapter classified Android component-recovery research by technique, evidence source, transformation threat, and evaluation practice. The synthesis shows a clear technical progression from package and profile matching toward graph-based, version-aware, optimization-aware, and learning-based methods. It also shows a persistent trust gap: detector outputs rarely preserve enough provenance, uncertainty, validation context, and security authority to become auditable reconstructed SBOM claims on their own.

The general conclusion returns to this gap through the research questions and future perspectives. The key lesson is that the reviewed techniques should not be judged only by whether they emit a name or version. They should also be judged by whether their outputs communicate why the recovered component entry should be believed, where its evidence stops, and how safely it may be used.

Part IV

General Conclusion

Chapter 4

General Conclusion and Perspectives

4.1 General Conclusion

Recovering a Software Bill of Materials from a compiled Android application is not only a detection problem. It is a problem of deciding what can be claimed from incomplete, transformed, and sometimes conflicting evidence. A component name or exact version may look precise while depending on weak namespace overlap, an incomplete reference collection, a feature-equivalent adjacent release, or source information that does not prove binary presence. Trust therefore depends on how evidence is qualified and bounded, not merely on whether a tool returns an answer.

This thesis studied that problem as a research question in Android third-party-library detection and binary SBOM reconstruction. The first chapter introduced the context, motivation, research questions, and scope of the work. Chapter 2 then established the technical background needed to understand software supply chains, SBOM standards, vulnerability identifiers, Android compilation, APK evidence, and the methodological constraints of a structured literature review. It showed why a compiled APK cannot be treated as a transparent dependency manifest and why binary evidence must be interpreted with provenance and uncertainty.

Chapter 3 organized the state of the art by recovery role, evidence source, transformation exposure, version capability, validation practice, and trust implication. The reviewed literature provides many useful signals: package and class profiles, structural graphs, fuzzy signatures, opcode and literal features, API references, learned representations, optimization-aware matching, and benchmark-driven assessment. However, the synthesis also showed recurring limitations. Exact-version identification is fragile under shrinking and optimization; reported scores are often tied to selected datasets; source-known dependencies are not always equivalent to binary presence; and most approaches stop before producing standard, provenance-aware SBOM output.

The central conclusion is therefore conditional. Android APK evidence can support useful reconstructed component claims, but only when the claim remains as precise as the evidence allows. Library identity and exact version must be distinguished. Missing reference coverage, near-tie versions, transformed code, native payloads, and dynamic loading must be visible rather than hidden. Vulnerability interpretation must inherit the

confidence of the component and version evidence instead of bypassing it. In this view, abstention and bounded claims are not failures; they are necessary protections against false precision.

4.2 Answers to the Research Questions

Table 4.1 summarizes the answers to the supporting research questions introduced in Chapter 1. The table is retained because it gives a compact link between the literature review and the final synthesis.

Table 4.1: Answers to the supporting research questions.

RQ	Question focus	Answer
RQ1	Evidence sources available in compiled Android applications	Recoverable evidence includes archive metadata, manifest information, DEX packages and classes, methods, opcodes, literals, API references, dependency and call relations, resources, native-file observations, reference artifacts, and external validation sources. These sources differ in directness, version specificity, resilience, and authority.
RQ2	How existing approaches combine evidence	Existing approaches range from namespace and class-set matching to structural graphs, opcode and semantic features, fuzzy signatures, learned representations, metadata coherence, and multi-stage reference matching. The literature supports multi-source reasoning, but no combination is universally reliable under shrinking, optimization, repackaging, feature-equivalent releases, and incomplete references.
RQ3	Factors that determine identity and version strength	Evidence strength depends on directness, uniqueness, independence, transformation resilience, version specificity, provenance reliability, reference coverage, and reproducibility. Identity and exact-version support must be assessed separately because a signal can identify a library family while remaining unable to distinguish adjacent releases.
RQ4	Representation of conflict, absence, transformation, and source dependence	These conditions should be represented through competing candidates, bounded version sets, conflict records, coverage limits, transformation exposure, report-only outcomes, and abstention. Missing evidence should not be interpreted as component absence, and source-known dependencies should not automatically be treated as proof of APK-side presence.

RQ	Question focus	Answer
RQ5	Information required for an auditable and security-relevant claim	A claim should identify the subject APK, candidate component and artifact family, claim type and granularity, version or version set, supporting and opposing evidence, provenance, coverage, transformation exposure, confidence class, ambiguity, validation context, and permitted downstream use. Vulnerability interpretation must inherit these boundaries.
RQ6	Evaluation without overstating accuracy	Evaluation should separate controlled fixtures, clean source-validated cases, partial source-validated cases, source-known upper bounds, diagnostics, report-only experiments, and unsupported limitations. It should publish denominators, distinguish emitted row correctness from full dependency recall, retain negative experiments and abstentions, and avoid promoting signals without control evidence.

Together, these answers shift evaluation from the question "Did the detector emit a version?" to the stronger question "What evidence supports this claim, what could invalidate it, and how may it safely be used?"

4.3 Future Works and Perspectives

Table 4.2: Priority perspectives for future Android APK-to-SBOM research.

Direction	Research objective	Required evidence
Empirical calibration	Calibrate identity, exact-version, ambiguity, and abstention classes by component family and transformation profile	Held-out ground truth, reliability analysis, and error-sensitive calibration metrics.
Independent validation	Apply recovery and reporting rules to other tools, analysts, and datasets	External tools, blinded case review, and inter-rater agreement on claim classes.
Public benchmark design	Measure exactness, coverage, abstention, provenance, and vulnerability-authority correctness together	Reproducible APKs, complete denominators, transformation labels, and version-discriminating ground truth.
Provenance-aware SBOM profiles	Standardize how reconstructed claims expose evidence, ambiguity, and validation context	Interoperable CycloneDX or related profiles tested across producers and consumers.

Direction	Research objective	Required evidence
Native and dynamic attribution	Extend analysis to shared libraries, protected payloads, and downloaded code	Separate corpora, architecture-aware fingerprints, runtime provenance, and scope-specific metrics.
Human factors and explainability	Determine whether analysts understand and act appropriately on claim classes, conflicts, and abstentions	User studies, decision-quality measures, and explanation comparisons.

Future work should first improve benchmark design. A useful benchmark should not only ask whether a detector found a name. It should record exact versions, transformed variants, reference coverage, native and dynamic boundaries, and the denominator used for each metric. Such a benchmark would make it possible to compare tools without confusing emitted-row precision with full dependency recall.

A second perspective is calibrated uncertainty. Instead of returning one forced exact version, future systems should support bounded version sets, confidence classes, and abstention policies whose empirical behavior is measured on independent data. This is especially important for adjacent versions, optimized code, and incomplete reference catalogs.

A third direction is interoperability. Reconstructed SBOM producers need a shared vocabulary for observed binary evidence, inferred identity, candidate version sets, external validation, and unsupported regions. CycloneDX extensions or profiles could preserve these distinctions while remaining usable by vulnerability-management tools.

Finally, Android research should expand beyond Maven bytecode. Native libraries, dynamically loaded code, protected payloads, and generated artifacts require their own evidence models and validation protocols. Treating them as ordinary Maven components would create false certainty. Treating them as explicit future research domains keeps the boundary honest while opening the path toward broader mobile software transparency.

Bibliography

[1] National Telecommunications and Information Administration. *The Minimum Elements for a Software Bill of Materials*. Tech. rep. U.S. Department of Commerce, 2021. URL: https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom.

[2] Jon Boyens et al. *Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations*. Tech. rep. NIST Special Publication 800-161 Revision 1. National Institute of Standards and Technology, 2022. DOI: 10.6028/NIST.SP.800-161r1. URL: https://csrc.nist.gov/pubs/sp/800/161/r1/final.

[3] Boming Xia et al. "An Empirical Study on Software Bill of Materials: Where We Stand and the Road Ahead." In: *arXiv preprint arXiv:2301.05362* (2023). DOI: 10.48550/arXiv.2301.05362.

[4] SPDX Workgroup. *SPDX Specifications*. URL: https://spdx.dev/use/specifications/.

[5] OWASP CycloneDX Project. *CycloneDX Specification*. URL: https://cyclonedx.org/specification/overview/.

[6] Package URL Project. *Package URL Specification*. URL: https://github.com/package-url/purl-spec.

[7] Android Developers. *Application Fundamentals*. URL: https://developer.android.com/guide/components/fundamentals.

[8] Android Developers. *App Manifest Overview*. URL: https://developer.android.com/guide/topics/manifest/manifest-intro.

[9] Android Developers. *Configure Your Build*. URL: https://developer.android.com/build.

[10] Gradle. *Variant-Aware Dependency Resolution*. URL: https://docs.gradle.org/current/userguide/variant_aware_resolution.html.

[11] Android Developers. *d8*. URL: https://developer.android.com/tools/d8.

[12] Android Developers. *Enable App Optimization with R8*. URL: https://developer.android.com/topic/performance/app-optimization/enable-app-optimization.

[13] Android Developers. *Analyze Your Build with the APK Analyzer*. URL: https://developer.android.com/studio/debug/apk-analyzer.

[14] Android Developers. *Enable Multidex for Apps with over 64K Methods*. URL: https://developer.android.com/build/multidex.

[15] Android Developers. *Create an Android Library*. URL: https://developer.android.com/studio/projects/android-library.

[16] OWASP Foundation. *Static Analysis*. URL: https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0014/.

[17] OWASP Foundation. *Reverse Engineering Android Apps*. URL: https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0013/.

[18] Android Developers. *Dynamic Code Loading*. URL: https://developer.android.com/privacy-and-security/risks/dynamic-code-loading.

[19] Luc Moreau and Paolo Missier. *PROV-DM: The PROV Data Model*. W3C Recommendation. World Wide Web Consortium, Apr. 2013. URL: https://www.w3.org/TR/prov-dm/.

[20] C. K. Chow. "On Optimum Recognition Error and Reject Tradeoff." In: *IEEE Transactions on Information Theory* 16.1 (1970), pp. 41–46. DOI: 10.1109/TIT.1970.1054406.

[21] Barbara Kitchenham and Stuart Charters. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Tech. rep. EBSE-2007-01. Keele University and Durham University, 2007. URL: https://www.elsevier.com/__data/promis_misc/525444systematicreviewsguide.pdf.

[22] Xian Zhan et al. "Automated Third-Party Library Detection for Android Applications: Are We There Yet?" In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 2020, pp. 919–930. DOI: 10.1145/3324884.3416582.

[23] Xian Zhan et al. "A Systematic Assessment on Android Third-Party Library Detection Tools." In: *arXiv preprint arXiv:2108.01964* (2021). DOI: 10.48550/arXiv.2108.01964.

[24] Michael Backes, Sven Bugiel, and Erik Derr. "Reliable Third-Party Library Detection in Android and Its Security Applications." In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 2016, pp. 356–367. DOI: 10.1145/2976749.2978333.

[25] Yan Wang et al. "ORLIS: Obfuscation-Resilient Library Detection for Android." In: *Proceedings of the 5th IEEE/ACM International Conference on Mobile Software Engineering and Systems*. 2018. DOI: 10.1145/3197231.3197248.

[26] Jiexin Zhang, Alastair R. Beresford, and Stephan A. Kollmann. "LibID: Reliable Identification of Obfuscated Third-Party Android Libraries." In: *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 2019. DOI: 10.1145/3293882.3330563.

[27] Menghao Li et al. "LibD: Scalable and Precise Third-Party Library Detection in Android Markets." In: *Proceedings of the 39th International Conference on Software Engineering*. 2017, pp. 335–346. DOI: 10.1109/ICSE.2017.38.

[28] Yuan Zhang et al. "Detecting Third-Party Libraries in Android Applications with High Precision and Recall." In: *Proceedings of the 25th IEEE International Conference on Software Analysis, Evolution and Reengineering*. 2018, pp. 141–152. DOI: 10.1109/SANER.2018.8330204.

[29] Xian Zhan et al. “ATVHUNTER: Reliable Version Detection of Third-Party Libraries for Vulnerability Identification in Android Applications.” In: *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*. 2021, pp. 1695–1707. DOI: 10.1109/ICSE43902.2021.00150.

[30] Yafei Wu et al. “LibScan: Towards More Precise Third-Party Library Identification for Android Applications.” In: *32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 3385–3402. ISBN: 978-1-939133-37-3. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/wu-yafei.

[31] Zifan Xie et al. “How Does Code Optimization Impact Third-Party Library Detection for Android Applications?” In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 2024. DOI: 10.1145/3691620.3695554.

[32] Bolin Zhou et al. “Version-Level Third-Party Library Detection in Android Applications via Class Structural Similarity.” In: *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. 2025, pp. 604–615. DOI: 10.1145/3756681.3756933.

[33] Dahan Pan et al. “Pay Your Attention on Lib! Android Third-Party Library Detection via Feature Language Model.” In: *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*. 2025, pp. 217–228. DOI: 10.1109/SANER64311.2025.00028.

[34] Pablo Marchand Ramirez. “Empirical Study of Third-Party Library Detection in iOS and Android Applications.” MA thesis. Universidad Politécnica de Madrid, 2024.

[35] Jintao Gu et al. “Revisiting Third-Party Library Detection: A Ground Truth Dataset and Its Implications Across Security Tasks.” In: *arXiv preprint arXiv:2509.04091* (2025). DOI: 10.48550/arXiv.2509.04091.

[36] Vadim Safronov et al. “UniBOM: A Unified SBOM Analysis and Visualisation Tool for IoT Systems and Beyond.” In: *arXiv preprint arXiv:2511.22359* (2025). DOI: 10.48550/arXiv.2511.22359.